A5: Performance, Test, and Bug Report

Barry Jiang
1008774097
barryy.jiang@mail.utoronto.ca

Kevin Liu
1008495237
xingjian.liu@mail.utoronto.ca

Markson Chen
1009002598
markson.chen@mail.utoronto.ca

Yibin Cui
1008826512
yibin.cui@mail.utoronto.ca

1 Part One — Profiling Execution Time

1.1 Overview

We profiled 5 important functions in our voice agent pipeline using Python's `cProfile` module:

#	Function	Component	What it does
1	`VAD._process_audio_frame`	Voice Activity Detection	Runs Silero VAD inference on each 20ms audio chunk
2	`ASR._transcribe_audio`	Speech-to-Text	Sends audio to Groq Whisper API for transcription
3	`LLM` TTFT via `litellm.completion`	Language Model	Calls Groq LLM API and streams tokens back
4	`TTS` TTFB via `requests.post`	Text-to-Speech	Calls Inworld TTS API and streams audio back
5	`Channel.send` / `_wait_and_get`	Inter-thread messaging	Passes data (frames) between pipeline components

Table 1: The 5 profiled functions.

Functions 1–4 form the audio pipeline that determines **Time to First Audio (TTFA)** — the latency from end-of-speech to first agent audio. Function 5 is the glue that connects them.

We first profile each audio pipeline function individually with cProfile (Section 1), then measure end-to-end TTFA in the full parallel pipeline (Section 2). Finally, we benchmark the Channel throughput against `threading.Queue` (Section 3).

Test Setup

- **Audio input**: Synthetic speech generated via macOS `say` (4.4s, 48kHz mono)
- **ASR**: Groq Whisper API (`whisper-large-v3-turbo`)
- **LLM**: Groq `llama-3.3-70b-versatile` (default), also tested OpenAI `gpt-5.4`
- **TTS**: Inworld TTS API (`inworld-tts-1.5-max`)

1.2 cProfile Analysis (per-function)

Each function was profiled individually with its own `cProfile` session, running components one at a time and feeding output → input sequentially.

1.2.1 `VAD._process_audio_frame` — 219 calls, 81ms total (0.4ms/call)

The VAD processes every 20ms audio chunk. The hot path is Silero VAD inference:

Function	Cumulative Time	Notes
`torch.nn.Module._call_impl`	60 ms	Silero VAD neural network forward pass
`AudioFrame.get` (resampling)	4 ms	numpy `interp` for 48kHz→16kHz
`torch.tensor`	4 ms	Converting audio chunks for Silero
`deque.popleft`	3 ms	VAD buffer management

In the pipeline, Smart Turn ONNX inference adds ∼13ms per chunk during silence detection, but the core VAD loop itself is very fast.

1.2.2 `ASR._transcribe_audio` — 1 call, 553ms

Dominated entirely by the Groq Whisper HTTP API call:

Function	Cumulative Time	Notes
`requests.post` → `sessions.send`	550 ms	Full HTTP round-trip
`ssl._SSLSocket.read`	465 ms	Waiting for Groq server response
`urllib3.connection.connect`	72 ms	TLS handshake
`_prepare_audio_for_transcription`	∼3 ms	WAV file creation + resampling

99% of time is network I/O.

1.2.3 `LLM` TTFT (`litellm.completion` → first token) — 540ms

cProfile captures the full `litellm.completion()` call up to the first streaming token:

Function	Cumulative Time	Notes
`litellm.utils.wrapper`	535 ms	litellm entry point
`token_counter` → `from_pretrained`	312 ms	Tokenizer loading (one-time cost)
`litellm.main.completion`	217 ms	Actual API call to Groq
`httpx.Client.send`	198 ms	HTTP request + first response chunk

Key finding: 312ms (58%) of the TTFT is litellm's tokenizer initialization (`from_pretrained`), not the actual API call. On subsequent calls this is cached and the real TTFT drops to ∼200ms.

1.2.4 `TTS requests.post` (Inworld API) — 522ms

Profiles the Inworld TTS API call including streaming audio response:

Function	Cumulative Time	Notes
`urllib3.connectionpool.urlopen`	492 ms	Full HTTP round-trip + streaming
`ssl._SSLSocket.read`	370 ms	Reading streaming audio chunks
`http.client._read_status`	368 ms	Waiting for first response byte
`urllib3.connection.connect`	122 ms	TLS handshake to Inworld
`create_connection`	74 ms	TCP socket setup

TLS handshake (122ms) is a significant fixed cost per request.

1.2.5 `Channel.send` / `Channel._wait_and_get` — 100K calls, 346ms total (3.5μs/call)

cProfile of the send path in a 1P1C throughput test (100K messages):

Function	Cumulative Time	Notes
`Sender.send`	262 ms	Entry point — iterates channels, tracks metrics
`Channel._send`	125 ms	Appends item + `notify_all()` under Condition lock
`Channel._gc`	56 ms	`min(cursors.values())` + `del items[:drop]` on every receive
`threading.notify_all`	40 ms	Wakes all waiting threads (Queue uses `notify` — wake one)
`sys.getsizeof`	8 ms	Byte count tracking on every send

The main overhead vs `threading.Queue`: `_gc()` runs on every receive (dict scan + list slice), and `notify_all()` wakes all threads instead of one. Queue's `put()`/`get()` is simpler: acquire lock → append to deque → notify one → release.

1.3 End-to-End Pipeline TTFA

After profiling each function individually, we ran the full pipeline in parallel to measure real TTFA:

`FileSource` → `VAD` → `ASR` → `LLM` → `TTS` → `NullSink`

TTFA is measured from the moment VAD finalizes the user's speech segment (end-of-utterance) to the first TTS audio chunk arriving at the sink. Each run is a separate process to avoid caching effects.

1.3.1 TTFA Breakdown (averaged over 10 pipeline runs)

Stage	Avg	Min	Max	What it measures
ASR (speech-to-text)	310 ms	169 ms	820 ms	VAD-end → ASR transcription complete
LLM TTFT (first token)	613 ms	453 ms	732 ms	ASR-done → first LLM token streamed
TTS TTFB (first audio)	406 ms	305 ms	589 ms	First LLM token → first TTS audio chunk
TTFA	**1,330 ms**	**1,033 ms**	**1,873 ms**	**VAD-end → first TTS audio (sum of above)**

1.3.2 Pipeline Hop Analysis

We confirmed there is no hidden overhead in the channel/thread infrastructure:

Hop	Latency
VAD finalize (segment concat)	1 ms
VAD → ASR channel transit	0 ms
ASR → Adapter → LLM channel transit	0 ms

1.3.3 LLM TTFT Comparison (Isolated)

We measured raw LLM TTFT with no pipeline overhead:

Model	Avg TTFT	Min	Max
Groq `llama-3.3-70b-versatile`	390 ms	257 ms	646 ms
OpenAI `gpt-5.4`	875 ms	458 ms	1,566 ms
OpenAI `gpt-4.1-nano`	1,157 ms	924 ms	1,437 ms

Groq is the fastest due to their custom LPU inference hardware.

1.3.4 Improvement Suggestions

1. `LLM` TTFT — Warm up tokenizer (613ms, 46% of TTFA) **Problem**: cProfile reveals 312ms of the 540ms is litellm loading the tokenizer via `from_pretrained`.

Improvement: Call `litellm.completion()` once at startup so the tokenizer is cached before real requests.

2. `TTS` `requests.post` — Pre-warm TLS connection (406ms, 31% of TTFA) **Problem**: TLS handshake to Inworld costs 122ms per request. Each TTS sentence creates a new connection.

Improvement: Use `requests.Session()` with connection pooling to reuse the TLS session across requests.

3. `ASR._transcribe_audio` — **Use streaming ASR (310ms, 23% of TTFA)** **Problem**: Each API call to Groq Whisper takes ∼550ms and processes the entire segment at once.

Improvement: Use a WebSocket-based ASR service (e.g., Deepgram) that transcribes incrementally as audio arrives.

4. `VAD._process_audio_frame` — **Reduce Smart Turn frequency (<5ms/chunk, minimal TTFA impact)** **Problem**: `_check_smart_turn()` runs ONNX inference (∼13ms) on every audio chunk during silence detection.

Improvement: Only run Smart Turn every 200ms instead of every 20ms. 5x/second is sufficient for turn detection.

1.3.5 Results After Implementing Improvements 1 & 2

We implemented the top two improvements:

1. **LLM tokenizer warmup**: Added `setup()` to `LLM` that calls `litellm.completion()` once at startup with a dummy prompt.
2. **TTS persistent session**: Replaced `requests.post()` with `self._session.post()` using a `requests.Session()`.

Stage	Before (avg)	After (avg)	Improvement
ASR	310 ms	280 ms	-30 ms
LLM TTFT	613 ms	448 ms	**-165 ms (27%)**
TTS TTFB	406 ms	427 ms	+21 ms (API variance)
TTFA	**1,330 ms**	**1,155 ms**	**-175 ms (13%)**

1.4 Channel Throughput Benchmark

We benchmarked our custom `Channel/Sender/Receiver` (function 5) against Python's `threading.Queue` using methodology adapted from the LMAX Disruptor perftest suite: 100K messages per run, 7 runs per test, `gc.collect()` between runs, correctness checksums on every run.

1.4.1 Throughput (median of 7 runs)

Topology	Channel	Queue	Ratio
1P1C (unicast)	512K ops/s	1,244K ops/s	Queue 2.4x faster
1P3C (multicast)	215K ops/s	414K ops/s	Queue 1.9x faster
3P1C (fan-in)	158K ops/s	1,188K ops/s	Queue 7.5x faster
Pipeline (P→S1→S2→S3)	120K ops/s	405K ops/s	Queue 3.4x faster

1.4.2 Latency (ping-pong round-trip, best of 5 runs)

Percentile	Channel	Queue
p50	8.4 μs	9.0 μs
p90	10.5 μs	9.6 μs
p99	13.8 μs	12.0 μs
p99.9	22.5 μs	23.3 μs
max	29.7 μs	103.0 μs

Channel has slightly lower median latency and better tail (30μs max vs 103μs).

1.4.3 Improvement: Rust Ring Buffer with CAS (FastChannel)

Problem: Channel is 2–7x slower than Queue on throughput. cProfile identifies `_gc()` on every receive, `notify_all()` on every send, and `sys.getsizeof()`/`time.time()` per-message metrics as the main overhead.

Improvement: Replace Channel's Python `list` + `threading.Condition` with a fixed-size ring buffer implemented in Rust via PyO3. The producer advances a write sequence counter via atomic `fetch_add` (CAS), and consumers advance their cursors via `compare_exchange` (CAS) — no mutex on the hot path. The ring buffer eliminates per-message GC (old slots are simply overwritten when the ring wraps).

1.4.4 Results After Implementing FastChannel

Topology	Channel	Queue	FastChannel
1P1C (unicast)	512K ops/s	1,244K ops/s	**2,921K ops/s**
1P3C (multicast)	215K ops/s	414K ops/s	**1,798K ops/s**
3P1C (fan-in)	158K ops/s	1,188K ops/s	**2,664K ops/s**
Pipeline (P→S1→S2→S3)	120K ops/s	405K ops/s	**981K ops/s**

FastChannel is **2–4x faster than Queue** and **5–8x faster than the original Channel**.

Percentile	Channel	Queue	FastChannel
p50	8.4 μs	9.0 μs	**7.5 μs**
p90	10.5 μs	9.6 μs	**8.0 μs**
p99	13.8 μs	12.0 μs	**10.4 μs**
p99.9	22.5 μs	23.3 μs	**17.6 μs**
max	29.7 μs	103.0 μs	**45.6 μs**

FastChannel has the lowest latency at every percentile.

2 Part Two

OpenNeuro measures coverage for both the backend and the frontend, and the results are published automatically through GitHub Actions.

2.1 Backend Tests and Coverage

The backend uses `pytest` together with `pytest-cov`. In `backend/pyproject.toml`, pytest is configured with `--cov=src`, `--cov-report=term-missing`, and `--cov-report=xml:coverage.xml`. This means every backend test run generates a machine-readable coverage report for the production code under `backend/src`.

The backend test suite currently contains 32 test files under `backend/tests`. These tests cover the API layer (`api`), application startup (`app`), core graph/channel/configuration logic (`core` and `test_graph_manager.py`), conduit modules such as ASR, TTS, memory, rendering, and streaming (`conduit`), and I/O behaviour such as sinks and source modules (`io`). Based on the local `pytest-cov` output provided for this report, backend coverage is **9555 / 9555 = 100.00%**.

One practical adjustment was made for CI: the GitHub Actions backend step skips `tests/conduit/dart_control`. Those DART tests are significantly slower than the rest of the suite, so excluding them from CI keeps workflow runtime reasonable and makes the automatic coverage job more stable. This does *not* remove DART testing from the project overall, because users can still run the full backend suite locally with `uv run pytest` or `bun run coverage:backend`, both of which include the DART tests.

2.2 Frontend Tests and Coverage

The frontend uses `Vitest` with Istanbul coverage. In `frontend/vite.config.ts`, coverage is enabled for all files in `src/**/*{ts,tsx}` with `all: true`, and the workflow exports an `lcov` report. The frontend

package also provides a dedicated command, `bun run coverage`, to execute the full UI test suite with coverage enabled.

The frontend currently contains 25 test files in `frontend/src`. These tests cover the main application entry points (`App.test.tsx`, `main.test.tsx`), graph UI components such as the canvas, nodes, edges, logging panel, metrics overlay, and configuration sidebar, shared UI widgets such as the dropdown, project and environment editors, React hooks, context state, and utility/API helper modules in `lib`. From the current `frontend/coverage/lcov.info` report, frontend line coverage is **1455 / 1455 = 100.00%**.

2.3 GitHub Workflow, README Badge, and PR Reporting

Coverage automation is implemented in `.github/workflows/coverage.yml`. This workflow runs on both pushes to `main` and pull requests targeting `main`. It sets up Python with `uv` for the backend and Bun for the frontend, runs `uv run pytest` in `backend`, then runs `bun run coverage` in `frontend`.

After both suites finish, the workflow runs `scripts/generate_coverage_badges.py`. This script reads `backend/coverage.xml` and `frontend/coverage/lcov.info`, generates the SVG badges `docs/backend-coverage.svg` and `docs/frontend-coverage.svg`, and also creates a Markdown coverage summary for pull requests. On pull requests, the workflow posts the summary as a sticky PR comment. On pushes to `main`, it commits the updated badge files automatically. The README already displays both badges at the top of the repository, so the latest backend and frontend coverage percentages are visible directly on the project homepage.

3 Part Three

Part Two shows that the project has broad test coverage across both major parts of the system.

For the backend, coverage breadth is high because tests are not limited to a single module; they span API controllers and services, application bootstrapping, the core graph runtime, utility code, multiple conduit pipelines, and source/sink I/O modules. For the frontend, the tests cover the application shell, graph editor components, metrics and logging views, project/environment configuration screens, shared hooks, context state management, and utility/helper functions. In other words, both the service layer and the user interface are included in coverage measurement rather than only a small set of isolated units.

Table 2: Local test coverage results used for grading

Suite	Covered / Total	% Stmts	% Branch	% Funcs	% Lines
Frontend	1455 / 1455	100%	100%	100%	100%
Backend	9555 / 9555	100%	N/A	N/A	100%

Because the coverage workflow runs automatically on both pull requests and pushes to `main`, this breadth is also continuously checked in CI. The backend tests, frontend tests, README badges, and PR coverage summary work together to make coverage visible and difficult to ignore during development.

4 Part Four: Test Coverage Depth

4.1 Module Choice: GraphManager

The module under test is `GraphManager` (`backend/src/core/graph.py`). It is the runtime orchestrator for OpenNeuro's backend — responsible for creating and destroying nodes, wiring edges between them, managing the lifecycle of channels, and starting/stopping the pipeline. Every mutation to the graph triggers a reconciliation algorithm that diffs the current channel layout against the desired one, reusing channels where possible to preserve in-flight data.

4.2 Responsibility, Interface, and Contract

Responsibility. GraphManager owns the full lifecycle of a directed dataflow graph. Components (audio sources, ML models, speakers, etc.) are nodes; edges declare which output slot feeds which input slot.

The manager translates this abstract graph into a running pipeline of threaded components connected by typed pub/sub channels.

Interface.

Method	Purpose
`add_node(type, init_args)`	Creates a component from the registry and adds it to the graph
`delete_node(node_id)`	Stops the component, removes all connected edges, stops affected downstream components, reconciles
`add_edge(edge)` / `delete_edge(edge)`	Adds or removes a connection, triggers channel reconciliation
`update_node(node_id, x, y)`	Updates node position (used by frontend)
`update_node_init_args(node_id, args)`	Hot-swaps a component's config—stops the pipeline, re-instantiates, restarts
`run()`	Wires Sender/Receiver handles into NamedTuples and starts all components
`stop()`	Idempotently stops all components
`reset(graph)`	Tears everything down and rebuilds from a new graph definition

Contract.

- Every node maps to exactly one Component instance. The `_graph.nodes` dict and `_components` dict must always stay in sync.
- Edges declare `(source_node, source_slot)` → `(target_node, target_slot)` connections. The channel layout is always minimal: receivers that share an identical set of senders are multiplexed onto a single Channel, guaranteeing the minimum number of channels for any given edge topology.
- When the graph mutates, `_reconcile()` diffs old vs new channel keys and reuses channels whose sender-set key hasn't changed—preserving any data already in the buffer.
- Deleting a node cascades: all edges touching that node are removed, affected downstream components are stopped, and a full reconcile runs.
- Adding an unknown node type raises `ValueError`. Deleting or updating a nonexistent node is a safe no-op (returns `None` or does nothing).
- `stop()` is idempotent—calling it multiple times does not crash.

4.3 The Reconciliation Algorithm

The core complexity of GraphManager lies in `_reconcile()`, which solves the following problem:

Problem statement. The pipeline graph induces a bipartite graph between two disjoint sets of slots. On one side are **sender slots** (output ports), identified by `(node_id, slot_name)`—these are the points where a component produces data. On the other side are **receiver slots** (input ports), also identified by `(node_id, slot_name)`—these are the points where a component consumes data. Each edge in the pipeline graph `(source_node, source_slot)` → `(target_node, target_slot)` corresponds to a connection from a sender slot to a receiver slot in this bipartite graph. The runtime must allocate **channels**—shared pub/sub buffers—to carry data between them. The goal is to produce a **minimal channel allocation**: the fewest channels such that every receiver's attached channel carries data from exactly the set of senders specified by the graph's edges—no more (which would introduce unwanted input) and no fewer (which would silently drop connections). Additionally, channels from the previous layout must be reused where possible to preserve in-flight data.

Algorithm. Reconciliation proceeds in three stages:

1. **Grouping (`_group`).** The edge list is inverted: for each receiver slot, collect the set of all sender slots that feed it. Receivers are then grouped by their *identical sender set*—if two receivers are fed by exactly the same set of senders, they can share a single channel. The group key is a `frozenset` of sender keys, making the grouping order-independent. This produces the minimum number of distinct channel keys for the given topology.

2. **Channel diffing.** The new set of group keys is compared against the existing `_channel_map`. Keys that appear in both old and new sets are *reused* (the same `Channel` object is kept, preserving any buffered data). Keys that are new get a fresh `Channel`. Keys that no longer appear are implicitly dropped (garbage collected).

3. **Handle reconciliation.** For each sender key, the algorithm collects all channels it must write to (a sender may fan out to multiple channels if it appears in multiple group keys). If the old `Sender` handle already points to the same set of channels, it is reused; otherwise a new `Sender` is created. The same logic applies to receivers: if the old `Receiver` handle already points to the correct channel, it is reused. Unconnected output slots still receive a no-op `Sender` (with zero channels) so that metrics remain visible.

Invariant. After reconciliation, the number of allocated channels equals the number of distinct sender-set groups—which is the theoretical minimum for the given edge topology. No two channels carry identical data, and no channel is unnecessarily recreated.

4.4 Tests and Edge Cases

#	Test	Type	Why this test?
1	`test_add_node`	Positive	Fundamental happy path—verifies that both `_graph.nodes` and `_components` are populated and in sync after adding a node.
2	`test_add_and_delete_edge`	Positive	Verifies that `_reconcile()` creates receiver/sender handles when an edge is added and cleans them up when removed.
3	`test_delete_node_removes_edges`	Positive	Cascade deletion—deleting a middle node must remove edges in both directions (incoming and outgoing), not just one.
4	`test_reconcile_reuses_channels`	Positive	Critical correctness property: adding an unrelated edge must not destroy existing channels, or in-flight data would be lost. Checks object identity (`is`).
5	`test_group_merges`	Positive	Core of the channel minimisation algorithm—two receivers fed by the same sender must be grouped under one channel key.
6	`test_group_separates_different_senders`	Positive	Converse of test 5—receivers with different sender sets must NOT share a channel, or data would leak across unrelated edges.
7	`test_update_node_position`	Positive	Simple setter, but verifies the return value is the same Node object in the graph (not a stale copy).
8	`test_build_namedtuple`	Positive	`_build_tuple` must construct a NamedTuple by keyword, matching field names to handle keys. Wrong construction causes TypeErrors at runtime.
9	`test_build_empty_for_none_type`	Positive	Source/sink components have `None` for their unused I/O type—must return `()`, not crash.
10	`test_build_empty_for_empty_handles`	Positive	Even with a valid type, if no handles are provided (no edges connected), must return `()`.
11	`test_run_and_stop`	Positive	Integration test—verifies components reach RUNNING status after `run()` and that `stop()` is idempotent (calling it twice doesn't crash).
12	`test_add_node_unknown_type`	Negative	Typos in node type names must raise `ValueError` immediately, not create a partially-initialised node. Also verifies the graph is unchanged after the error.

#	Test	Type	Why this test?
13	`test_delete_nonexistent_node`	Negative	In a concurrent UI, delete requests can arrive for already-removed nodes. Must not crash or affect existing nodes.
14	`test_update_nonexistent_node_position`	Negative	Stale node references must return `None`, not KeyError.
15	`test_update_nonexistent_node_init_args`	Negative	Same as above but for the hot-reload path, which has more complex logic (registry lookup, stop/restart).

4.5 Link to Tests

`backend/tests/test_graph_manager.py`

5 Part Five

`https://youtu.be/G_RB7JvWcJM`